

Stage 4 实验报告

2022011223 熊泽恩

实验内容

为了完成这次实验，我进行了以下工作：

- 在 `frontend/ast/tree.py` 中添加了 `For` 和 `Continue` 两个类，同时在 `frontend/ast/visitor.py` 中补充了对应的 `visitor` 成员函数。
- 在 `frontend/lexer/lex.py` 和 `frontend/parser/ply_parser.py` 中添加了 `for` 循环和 `break` 的 `token` 定义和语法规则，使得它们能够被正确识别。
- 修改了 `frontend/scope/scope.py` 和 `frontend/scope/scopestack.py`，为每个 `scope` 添加了 `loop_count` 成员变量，用于记录当前作用域位于多少个循环里。
- 修改了 `frontend/tacgen/tacgen.py`：
 - 补全了 `visitFor` 函数，实现了 `for` 循环的中间代码生成，包括 `init`、`cond`、`body` 和 `update` 四个部分。
 - 补全了 `visitCondExpr` 函数，完成了从三目运算符到中间代码的转换。和 `if-else` 语句不同的一点是，三目运算符是有返回值的，所以需要新建一个临时变量用于存 `then` 或 `otherwise` 之一的返回值。
 - 重载了 `visitContinue` 函数，保证 `continue` 语句的跳转是正常的。最开始我没重载这个函数，发现 `continue` 语句的有无，对于最后的三地址码结果没有影响（直接忽视了 `continue` 语句的存在）。后来发现需要重载这个函数，加上就通过所有测试了。
- 修改了 `frontend/typecheck/namer.py`：
 - 补全了 `Namer.visitCondExpr` 函数，按照和 `if` 一样的顺序，分别访问 `cond`、`then` 和 `otherwise`。
 - 补全了 `Namer.visitFor` 函数。我专门特判了 `stmt.cond` 为空的情况，此时按照实验指导书上写的，应该直接将空的 `cond` 看做非零数字常量，所以我将其赋值为 `IntLiteral(1)`。
 - 修改了 `visitwhile` 和 `Namer.visitBreak` 函数，补全了 `Namer.visitContinue` 函数，保证在进入 `while` 循环的时候，会将栈顶的作用域的 `loop_count` 加一。如果栈顶作用域的 `loop_count` 为 0，说明当前不在任何循环中，若此时再 `break` 或 `continue`，则会报错。

Stage 4 思考题

Step 7

第 1 题

我们的实验框架里是如何处理悬吊 `else` 问题的？请简要描述。

在 `frontend/parser/ply_parser.py` 中，有两种 `statement` 的定义：一种是 `statement_matched`，一种是 `statement_unmatched`，它们分别表示 `if-else` 完全匹配，一种表示只有 `if` 没有 `else`。

设 `statement_matched` 为 A ，`statement_unmatched` 为 B ，`expression` 为 E ，则对应的 CNF 如下：

$$\begin{aligned}
 A &\rightarrow \text{if } (E) \ A \text{ else } A \\
 B &\rightarrow \text{if } (E) \ A \text{ else } B \\
 B &\rightarrow \text{if } (E) \ A \\
 &\dots
 \end{aligned}$$

所以，实验框架在遇到以下情况时，`else d = 0;` 会与 `if (b)` 匹配，而不是与 `if (a)` 匹配。

```

1  if (a)
2      if (b)
3          c = 0;
4      else
5          d = 0;

```

解析第一个 `if (a)` 时，会默认它没有 `else` 分支，因为 `else` 语句尚未出现。然而，当解析器遇到第二个 `if (b)` 时，它必须选择一个 `else` 语句与之匹配，这时它会选择最近的一个没有匹配的 `else`，即 `else d = 0;`。

第 2 题

在实验要求的语义规范中，条件表达式存在短路现象。即：

```

1  int main() {
2      int a = 0;
3      int b = 1 ? 1 : (a = 2);
4      return a;
5  }

```

会返回 0 而不是 2。如果要求条件表达式不短路，在你的实现中该做何种修改？简述你的思路。

如果在 `frontend/tacgen/tacgen.py` 里，将 `expr.otherwise.accept(self, mv)` 放到 `expr.then.accept(self, mv)` 之后，`mv.visitLabel(exitLabel)` 之前，这样就能实现条件表达式不短路执行。

这是因为，在条件为真的情况下，“执行 otherwise 部分的语句”位于“跳到 if 语句结束”之前。此时，条件表达式不短路执行。

Step 8

第 1 题

将循环语句翻译成 IR 有许多可行的翻译方法，例如 `while` 循环可以有以下两种翻译方式：

第一种（即实验指导中的翻译方式）：

- `label BEGINLOOP_LABEL`：开始新一轮迭代
- `cond` 的 IR
- `beqz BREAK_LABEL`：条件不满足就终止循环
- `body` 的 IR
- `label CONTINUE_LABEL`：continue 跳到这
- `br BEGINLOOP_LABEL`：本轮迭代完成
- `label BREAK_LABEL`：条件不满足，或者 break 语句都会跳到这儿

第二种:

- `cond` 的 IR
- `beqz BREAK_LABEL`: 条件不满足就终止循环
- `label BEGINLOOP_LABEL`: 开始新一轮迭代
- `body` 的 IR
- `label CONTINUE_LABEL`: continue 跳到这
- `cond` 的 IR
- `bnez BEGINLOOP_LABEL`: 本轮迭代完成, 条件满足时进行下一次迭代
- `label BREAK_LABEL`: 条件不满足, 或者 `break` 语句都会跳到这儿

从执行的指令的条数这个角度 (`label` 不算做指令, 假设循环体至少执行了一次), 请评价这两种翻译方式哪一种更好?

如果不包括循环体的指令, 则

- 第一种方式: 每次迭代固定执行 5 条指令。
- 第二种方式: 第一次迭代执行 3 条指令, 之后每次迭代执行 6 条指令。

从执行的指令条数这个角度来看, 第一种翻译方式更好, 因为对于循环中的每一次迭代而言, 第一种方式相较于第二种方式总会少执行一条指令。这种方式可以减少总的指令执行数, 从而提高程序的运行效率。

第 2 题

我们目前的 TAC IR 中条件分支指令采用了单分支目标 (标签) 的设计, 即该指令的操作数中只有一个是标签; 如果相应的分支条件不满足, 则执行流会继续向下执行。在其它 IR 中存在双目标分支 (标签) 的条件分支指令, 其形式如下:

```
1 | br cond, false_target, true_target
```

其中 `cond` 是一个临时变量, `false_target` 和 `true_target` 是标签。其语义为: 如果 `cond` 的值为 0 (假), 则跳转到 `false_target` 处; 若 `cond` 非 0 (真), 则跳转到 `true_target` 处。它与我们的条件分支指令的区别在于执行流总是会跳转到两个标签中的一个。

你认为中间表示的哪种条件分支指令设计 (单目标 vs 双目标) 更合理? 为什么? (言之有理即可)

我觉得双目标更合理, 因为双目标分支在某些情况下可以减少代码大小, 而且能使代码可读性更高。

比如, 它可以将两个跳转合并为一个指令。在下面这份代码中:

```
1 | if (cond) {
2 |     then
3 | } else {
4 |     otherwise
5 | }
```

假设 `then` 和 `otherwise` 的语句的代码块分别对应标签 `L1` 和 `L2`, 则可以用双目标分支写 TAC 如下:

```
1 br cond, L1, L2
2 L1:
3     ...
4 L2:
5     ...
```

这样能够使代码长度大大缩短，更加简洁易懂。